# Order Flow - Coding challenge

Build a small order-fulfilment system using three services.
The domain itself is deliberately simple, I'm not looking for fancy features or a polished UI. What I care about is how you handle **distributed state across service boundaries**.

**Goal:** Show both a successful order and a payment failure, with stock correctly restored in the failure case.

I would ask you to use the following tech stack for this exercise:

- C# / .NET 10
- Microservices architecture
- Apache Pulsar
- PostgreSQL or MySQL (one database per service)
- Blazor WebAssembly
- Docker & docker-compose

## 1. I want you to get familiar with these terminologies below:

| Term | Meaning |
|---|---|
| **Order** | A customer's purchase request containing one or more lines (SKU + quantity + unit price). |
| **SKU** | Stock-keeping unit – a product identifier such as WIDGET-01. |
| **Reservation** | A temporary hold on stock for a specific order. The hold lasts until the order is either confirmed or cancelled. |
| **Choreographed saga** | A distributed transaction where each service reacts to events and publishes its own. There is no central coordinator; compensating actions are triggered by events. |
| **Compensation** | An action that undoes a previous step (for example, releasing reserved stock). This is not a database rollback. |
| **Transactional outbox** | Writing the outgoing event into a table in the same database transaction as the business state change, then publishing it asynchronously. |
| **Inbox / dedup** | Storing the IDs of already-processed messages so that redelivered messages can be ignored. |
| **DLQ** | Dead-letter topic – where messages go after they have failed delivery too many times. |
| **Partition key** | The value Pulsar uses to keep related messages on the same partition, preserving their order. |

## 2. Expected workflow

1. A customer places an order through the Orders service. The order is created with status **Pending**.
2. The Orders service publishes an **OrderPlaced** event.

3. The Inventory service tries to reserve stock for every line in the order.
    - If everything can be reserved → it publishes **ReservationSucceeded**.
    - If any line cannot be reserved → it publishes **ReservationFailed**.
4. The Orders service reacts:
    - **ReservationSucceeded** → status becomes **Charging**.
    - **ReservationFailed** → status becomes **Cancelled**.
5. The Payments service listens for **ReservationSucceeded** and attempts to charge the customer (using a fake payment gateway).
    - Success → publishes **PaymentSucceeded** → Orders moves the order to **Confirmed**.
    - Failure → publishes **PaymentFailed** → Inventory releases the reservation (compensation) → Orders moves the order to Cancelled.
6. After a successful payment, Inventory must permanently take the reserved stock out of the available pool (either by reducing **quantity_on_hand** or by marking the reservation as **Consumed**). Please document which approach you chose.
7. After a payment failure, stock levels must return **exactly** to what they were before the order. No stock may be lost or double-counted at any point.

# 3. Expected services

You need three backend services, each with its own database, plus a Blazor WebAssembly client.
The ports listed below are the ones exposed on the host by **docker compose**.

**Common ownership rules**

- A service may only read and write its own database. No cross-database queries, no shared **DbContext**, and no foreign keys that reach into another service's schema.
- IDs that belong to other services (**OrderId**, **Sku**, etc.) are stored as plain values.
- Every service must expose **GET /health** and return 200 only when both its database and its Pulsar connection are healthy.
- Configuration comes exclusively from environment variables (the variable names used in **docker-compose.yml** are fixed).

**Order identity**
Use a single GUID for **orderId / correlationId** everywhere. Do not mix numeric database IDs with GUIDs in the events.

## 3.1 Orders Service - *http://localhost:5001*

Owns the Order aggregate and is the source of truth for the overall progress of the saga.

**Tables (schema orderflow_orders)**

| Table | Purpose |
|---|---|
| orders | Order header (customer, total, status, timestamps). |
| order_lines | The individual lines belonging to an order. |
| order_saga_state | Tracks which steps of the saga have completed (used by GET /orders/{id}). |
| outbox_messages | Transactional outbox. |

| Table | Purpose |
| --- | --- |
| inbox_messages | Deduplication store. |

**Order status progression**
**Pending** → **Reserving** → **Charging** → **Confirmed**
**Cancelled** can be reached from either **Reserving** or **Charging**.

**Endpoints**

**POST /orders**

```
// Request
{
  "customerId": "cust-123",
  "lines": [
    { "sku": "WIDGET-01", "quantity": 2, "unitPrice": 10.00 },
    { "sku": "WIDGET-02", "quantity": 1, "unitPrice": 5.50 }
  ]
}

// Response 202 Accepted
{
  "orderId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "correlationId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "status": "Pending"
}
```

**GET /orders/{id}**

```
{
  "orderId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "customerId": "cust-123",
  "status": "Charging",
  "totalAmount": 25.50,
  "reservationCompleted": true,
  "paymentCompleted": false,
  "lines": [
    { "sku": "WIDGET-01", "quantity": 2, "unitPrice": 10.00 },
    { "sku": "WIDGET-02", "quantity": 1, "unitPrice": 5.50 }
  ],
  "createdAt": "2026-09-01T10:15:00Z",
  "updatedAt": "2026-09-01T10:15:12Z"
}
```

**GET /orders?customerId={customerId}**
Returns a list of the customer's orders (id, status, total, createdAt is enough).

**GET /health** – 200 when database and Pulsar are both reachable.

**Events**

- Publishes: **OrderPlaced** (and optionally **OrderCancelled**)
- Subscribes: **ReservationSucceeded**, **ReservationFailed**, **PaymentSucceeded**, **PaymentFailed**

## 3.2 Inventory Service - *http://localhost:5002*

Owns stock levels and reservations.

**Tables (schema orderflow_inventory)**

| Table | Purpose |
|---|---|
| stock_items | Current on-hand and reserved quantities per SKU. |
| reservations | One row per reserved order line (Active / Released / Consumed). |
| outbox_messages | Outbox. |
| inbox_messages | Deduplication. |

**Endpoints**

**GET /stock**

```
[
 {
  "sku": "WIDGET-01",
  "quantityOnHand": 10,
  "quantityReserved": 2,
  "available": 8
 },
 {
  "sku": "WIDGET-02",
  "quantityOnHand": 5,
  "quantityReserved": 0,
  "available": 5
 }
]
```

**POST /stock/{sku}/adjust** (demo helper only)

```
// POST /stock/WIDGET-01/adjust
{ "quantity": 20 }

// Response
{
 "sku": "WIDGET-01",
 "quantityOnHand": 30,
 "quantityReserved": 0,
 "available": 30
}
```

**GET /health**

**Events**

- Subscribes: OrderPlaced, PaymentSucceeded, PaymentFailed
- Publishes: ReservationSucceeded, ReservationFailed, and optionally StockReleased

**Concurrency**
Reserving stock must be atomic. Two concurrent requests for the last remaining unit must not both succeed. Use a conditional UPDATE or SELECT … FOR UPDATE (or an equivalent safe approach). Personally I would prefer utilizing Entity Framework Core.

### 3.3 Payments Service - *http://localhost:5003*

Owns payments. There is no real payment gateway, just a fake one.

**Tables (schema orderflow_payments)**

| Table | Purpose |
|---|---|
| payments | One row per payment attempt. |
| outbox_messages | Outbox. |
| inbox_messages | Deduplication. |

**Endpoints**

**GET /payments/{orderId}**

```
{
  "paymentId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "orderId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "amount": 25.50,
  "status": "Succeeded",
  "createdAt": "2026-09-01T10:15:15Z"
}
```

Return 404 if no payment exists yet.

**GET /health**

**Fake gateway rule**
The gateway must fail whenever the order total ends in **.99** (e.g**. 19.99**). This makes the failure scenario easy and deterministic to demo.

**Events**

- Subscribes: **ReservationSucceeded**
- Publishes: **PaymentSucceeded**, **PaymentFailed**

### 3.4 Blazor WebAssembly Client - *http://localhost:5000*

Two simple pages are enough:

1. **Place order**: choose SKUs and quantities, submit, then poll the order status every couple of seconds so you can watch the saga progress.
2. **Dashboard**: show recent orders together with current stock levels so a reviewer can see stock being reserved and later released.

Because the client is a static SPA it cannot read environment variables at runtime. Put the API base URLs in **appsettings.json** (or the equivalent). No connection strings or secrets should live in the client.

## 4. Event contracts

Every event must carry at least:

- **eventId** (unique GUID)
- **correlationId** / **orderId** (the same GUID)
- **timestamp** (UTC)

Use **orderId** as the Pulsar partition key so that all events belonging to one order are processed in order.

| Event | Publisher | Subscribers | Suggested payload |
|---|---|---|---|
| OrderPlaced | Orders | Inventory | orderId, customerId, lines, totalAmount |
| ReservationSucceeded | Inventory | Orders, Payments | orderId, reservationId, lines |
| ReservationFailed | Inventory | Orders | orderId, reason |
| PaymentSucceeded | Payments | Orders, Inventory | orderId, paymentId, amount |
| PaymentFailed | Payments | Orders, Inventory | orderId, reason |
| StockReleased | Inventory | Orders (optional) | orderId, releasedLines |

Feel free to add extra events if they help, as long as the contracts stay consistent.

# 5. Requirements

- Each service owns its own PostgreSQL/ MySQL database. No service is allowed to read or write another service's tables.
- Communication inside the saga must be done only through Pulsar events. Synchronous HTTP calls between the services are not allowed as part of the transaction.
- Any state change that results in an event must be performed in the same database transaction as the insertion into the outbox table (transactional outbox pattern).
- Consumers have to be idempotent. Pulsar is at-least-once, so the same event may arrive more than once; using an inbox table (or equivalent) to ignore duplicates is expected.
- Events for a single order must be processed in order, use the **orderId** as the partition key.
- Compensation has to be real. After a payment failure the stock figures must be exactly the same as they were before the order started, and the order must end up in **Cancelled**.
- Concurrent attempts to reserve the last available unit must not both succeed.
- Poison messages should not block the whole pipeline. Configure a dead-letter topic and move a message there after **three failed delivery attempts.**
- **docker compose up** should bring everything up (Postgres/ MySQL, Pulsar, the three services, schema creation and seed data).
- All configuration for the backend services comes from environment variables. The Blazor client must not contain secrets or connection strings.
- After a successful payment the Inventory service must permanently consume the reserved stock. Mention in the README which approach you picked.

# 6. Rules

- You may use any NuGet package except a full-blown saga / workflow framework (MassTransit sagas, NServiceBus, Dapr Workflow, etc.). I want to see how you reason about distributed state yourself. A thin wrapper around the Pulsar client is fine.
- AI tools are not suggested, I expect that you must be able to explain every part of the solution in the review.
- Questions are welcome - asking good clarifying questions early is a plus.

# 7. Suggested database schema

You are free to design the tables however you like. The snippets below are only a starting point.

## Orders (orderflow_orders)

```
CREATE TABLE orders (
   id            UUID PRIMARY KEY,
   customer_id     VARCHAR(100) NOT NULL,
   total_amount    NUMERIC(10,2) NOT NULL,
   status         TEXT NOT NULL CHECK (status IN ('Pending','Reserving','Charging','Confirmed','Cancelled')),
   created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
   updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE order_lines (
   id            BIGSERIAL PRIMARY KEY,
   order_id       UUID NOT NULL REFERENCES orders(id),
   sku           VARCHAR(50) NOT NULL,
   quantity       INT NOT NULL,
   unit_price      NUMERIC(10,2) NOT NULL
);

CREATE TABLE order_saga_state (
   order_id                UUID PRIMARY KEY REFERENCES orders(id),
   reservation_completed      BOOLEAN NOT NULL DEFAULT FALSE,
   payment_completed          BOOLEAN NOT NULL DEFAULT FALSE,
   last_processed_event_id     UUID NULL
);

CREATE TABLE outbox_messages (
   id            BIGSERIAL PRIMARY KEY,
   event_id       UUID NOT NULL UNIQUE,
   topic          VARCHAR(255) NOT NULL,
   payload        JSONB NOT NULL,
   created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
   published_at    TIMESTAMPTZ NULL
);

CREATE TABLE inbox_messages (
   event_id       UUID PRIMARY KEY,
   processed_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

## Inventory (orderflow_inventory)

```
CREATE TABLE stock_items (
   sku              VARCHAR(50) PRIMARY KEY,
   quantity_on_hand    INT NOT NULL,
   quantity_reserved   INT NOT NULL DEFAULT 0
);
```

```
CREATE TABLE reservations (
   id             BIGSERIAL PRIMARY KEY,
   order_id       UUID NOT NULL,
   sku            VARCHAR(50) NOT NULL,
   quantity       INT NOT NULL,
   status         TEXT NOT NULL CHECK (status IN ('Active','Released','Consumed')),
   created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
   UNIQUE (order_id, sku)
);

-- outbox_messages and inbox_messages same shape as in Orders
```

## Payments (orderflow_payments)

```
CREATE TABLE payments (
   id             UUID PRIMARY KEY,
   order_id       UUID NOT NULL UNIQUE,
   amount         NUMERIC(10,2) NOT NULL,
   status         TEXT NOT NULL CHECK (status IN ('Succeeded','Failed')),
   created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- outbox_messages and inbox_messages same shape as in Orders
```